

Báo cáo Nghiên cứu Thực nghiệm: Tối ưu hóa Chiến lược Chunking cho Hệ thống RAG miCareer-x-Fang

Tổng quan Kiến trúc và Tầm quan trọng của Chiến lược Phân mảnh Dữ liệu

Hệ thống miCareer được định vị không chỉ là một nền tảng kết nối việc làm mà còn là một hệ thống quản trị tuyển dụng thu nhỏ (mini-ATS) toàn diện, bao quát toàn bộ vòng đời của một ứng viên từ giai đoạn nộp hồ sơ, phỏng vấn, cho đến khi đàm phán hợp đồng.¹ Sự phức tạp của hệ thống ATS hiện đại nằm ở khối lượng khổng lồ các dữ liệu phi cấu trúc mang giá trị thông tin cao, điển hình như Hồ sơ ứng viên (CV), Mô tả công việc (Job Description - JD), và Thư xin việc (Cover Letter). Nhằm giải quyết bài toán sàng lọc hồ sơ tốn kém thời gian và dễ mang định kiến chủ quan của con người, kiến trúc Retrieval-Augmented Generation (RAG) đã được tích hợp vào hệ thống thông qua một lớp trí tuệ nhân tạo độc lập mang tên Fang.¹

Trong kiến trúc RAG, toàn bộ sự thành bại của hệ thống phụ thuộc vào chất lượng của giai đoạn tiếp nhận và xử lý dữ liệu đầu vào (Ingestion). Giai đoạn này bao gồm một chuỗi các thao tác: trích xuất văn bản (parse), phân mảnh dữ liệu (chunk), nhúng vector (embed), và lưu trữ (store).¹ Mặc dù các mô hình ngôn ngữ lớn (LLM) hiện đại sở hữu khả năng suy luận vượt trội, chúng hoàn toàn bị vô hiệu hóa nếu hệ thống truy xuất (Retriever) cung cấp cho chúng những đoạn văn bản bị cắt xén sai ngữ cảnh, mất logic, hoặc thiếu các thông tin định danh quan trọng.³ Do đó, chiến lược chunking không đơn thuần là một bước tiền xử lý kỹ thuật, mà là một quyết định kiến trúc cốt lõi quyết định trực tiếp đến độ phủ thông tin (Recall), độ chính xác (Precision) và chi phí vận hành của toàn bộ hệ thống RAG.³

Mục tiêu cốt lõi của báo cáo nghiên cứu này là tiến hành phương pháp cắt bỏ (ablation study) trên ba chiến lược chunking phổ biến: Token-based (Dựa trên số lượng token), Recursive (Đệ quy cấu trúc văn bản), và Semantic/Structure-aware (Nhận thức cấu trúc và ngữ nghĩa). Thông qua các thử nghiệm thực tế trên tập dữ liệu tuyển dụng của dự án miCareer, kết hợp cùng kỹ thuật ghim bối cảnh (Section-Pinning), báo cáo sẽ xác định điểm cân bằng tối ưu (sweet-spot) giữa hiệu suất truy xuất và chi phí lưu trữ trên cơ sở dữ liệu PostgreSQL.

Nền tảng Lưu trữ Vector và Không gian Ngữ nghĩa

Trước khi đi sâu vào các chiến lược phân mảnh, việc hiểu rõ cơ sở hạ tầng lưu trữ và toán học đằng sau quá trình truy xuất là điều kiện tiên quyết để giải thích các hiện tượng quan sát được

trong thực nghiệm.

Vai trò của PostgreSQL và Extension pgvector

Thay vì sử dụng các cơ sở dữ liệu vector chuyên biệt và đắt đỏ (như Pinecone hay Milvus), hệ thống miCareer-x-Fang lựa chọn tận dụng sức mạnh của PostgreSQL (phiên bản 18.3) kết hợp cùng extension mã nguồn mở pgvector (phiên bản 0.8.2).¹ Quyết định kiến trúc này cho phép lưu trữ dữ liệu nghiệp vụ truyền thống và các vector đa chiều trong cùng một hệ sinh thái, duy trì tính toàn vẹn của chuẩn hóa 3NF và loại bỏ độ trễ giao tiếp giữa các hệ thống cơ sở dữ liệu phân tán.¹

Dữ liệu văn bản sau khi được phân mảnh sẽ được chuyển đổi thành các vector thông qua mô hình nhúng (embedding model). Mỗi vector đại diện cho một điểm trong không gian toán học nhiều chiều, nơi các đoạn văn bản mang ý nghĩa tương đồng sẽ được xếp gần nhau.¹ Cấu trúc này mở ra khả năng tìm kiếm ngữ nghĩa (semantic search), vượt qua những hạn chế cơ bản của tìm kiếm từ khóa truyền thống.¹

Các Phép đo Khoảng cách và Thuật toán Chỉ mục

Hệ thống truy xuất sử dụng hai phép đo khoảng cách chính để đánh giá mức độ tương đồng giữa câu hỏi của nhà tuyển dụng (query vector) và các đoạn tài liệu (document vectors):

- Khoảng cách L2 (Euclidean Distance - Toán tử <=>): Đo lường khoảng cách hình học tuyến tính tuyệt đối giữa hai điểm trong không gian. Phép đo này nhạy cảm với độ lớn (magnitude) của vector.¹
- Khoảng cách Cosine (Cosine Distance - Toán tử <=>): Đo lường góc giữa hai vector, bỏ qua sự khác biệt về độ dài của văn bản gốc. Đối với bài toán xử lý ngôn ngữ tự nhiên, khoảng cách Cosine thường mang lại hiệu suất vượt trội vì nó tập trung hoàn toàn vào sự tương đồng về mặt hướng đi (ngữ nghĩa) của thông tin.¹

Để xử lý hàng triệu vector trong thời gian thực, hệ thống triển khai các thuật toán chỉ mục nhằm tránh việc phải quét toàn bộ bảng dữ liệu (exact nearest neighbor search).¹ Hai thuật toán được xem xét bao gồm IVFFlat (Inverted File Index) và HNSW (Hierarchical Navigable Small World). Thuật toán IVFFlat hoạt động bằng cách phân cụm không gian vector thành các vùng, giúp hệ thống chỉ cần tìm kiếm trong các cụm lân cận nhất với truy vấn. Mặc dù tối ưu về bộ nhớ, IVFFlat dễ rơi vào tình trạng bỏ sót các kết quả nằm ở rìa các cụm.¹ Ngược lại, HNSW xây dựng một mạng lưới đồ thị đa tầng. Khi có truy vấn, thuật toán sẽ duyệt từ các tầng thưa thớt ở trên cùng để nhanh chóng xác định khu vực tiềm năng, sau đó đi sâu xuống các tầng dày đặc bên dưới để định vị chính xác vector mục tiêu. Phương pháp đồ thị này đảm bảo tốc độ truy xuất cực nhanh và độ chính xác gần như tuyệt đối, trở thành lựa chọn ưu tiên cho kiến trúc RAG của dự án bất chấp việc tiêu tốn nhiều dung lượng RAM hơn.¹

Cấu trúc Luồng Dữ liệu Đầu vào (Ingestion Pipeline)

Tính đặc thù của miền dữ liệu tuyển dụng đòi hỏi một luồng xử lý đầu vào phức tạp hơn so với các hệ thống hỏi đáp tài liệu thông thường. Hệ thống miCareer xử lý ba nguồn dữ liệu văn bản chính: Mô tả công việc, Thư xin việc, và Hồ sơ ứng viên.

Đặc tính của JD và Cover Letter

Mô tả công việc (JobPosting.desc) và Thư xin việc (coverLetter) thường mang cấu trúc văn bản trần thuật hoặc các đoạn văn bản dài có tính liên kết chặt chẽ.¹ JD được phân chia rõ ràng bằng các tiêu đề như "Yêu cầu công việc", "Trách nhiệm", và "Quyền lợi", trong khi Cover Letter trình bày mạch lạc về động lực và sự phù hợp của ứng viên với vị trí.¹ Việc trích xuất văn bản từ hai nguồn này tương đối đơn giản vì chúng thường được lưu trữ trực tiếp dưới dạng văn bản thô hoặc HTML cơ bản trong cơ sở dữ liệu.¹

Độ phức tạp của CV và Hệ thống Trích xuất (Parser)

Khác với JD hay Cover Letter, Hồ sơ ứng viên (CV) là một bản tóm tắt thông tin mang đậm tính cá nhân hóa, chứa đựng vô số các định dạng phức tạp như bố cục nhiều cột, bảng biểu, biểu tượng, và thậm chí là các thanh tiến trình (progress bars) để đánh giá kỹ năng.¹¹ Một CV có thể được nộp dưới định dạng PDF có chứa lớp văn bản (text layer), DOCX, hoặc một file ảnh quét (scanned image).

Để giải quyết sự đa dạng này, hệ thống triển khai một đường ống trích xuất đa lớp. Đối với các file PDF tiêu chuẩn, thư viện pdfplumber hoặc PyMuPDF được sử dụng để bóc tách văn bản, đồng thời thu thập các siêu dữ liệu (metadata) về tọa độ, phông chữ, và kích thước chữ nhằm nhận diện các tiêu đề phân mục (Section Headers).¹¹ Trong trường hợp hệ thống phát hiện một trang tài liệu là hình ảnh hoặc quá trình trích xuất văn bản nguyên bản thất bại, công cụ nhận dạng ký tự quang học Tesseract OCR sẽ được kích hoạt làm phương án dự phòng để chuyển đổi pixel thành văn bản.¹¹

Kết quả của toàn bộ quá trình này được lưu trữ vào trường CVPARSED.rawText dưới dạng một đối tượng JSON có cấu trúc.¹ Khác với một chuỗi văn bản dài thông thường, đối tượng JSON này bảo toàn các trường thông tin (fields) quan trọng, giúp hệ thống phân định rõ ràng ranh giới giữa mục Học vấn, Kinh nghiệm làm việc, Kỹ năng, và Thông tin cá nhân. Sự chuẩn bị kỹ lưỡng này là nền tảng tối quan trọng cho các chiến lược chunking nhận thức cấu trúc sẽ được phân tích ở các phần tiếp theo.¹

Khung Lý thuyết và Phương pháp Cắt bỏ (Ablation Methods)

Để đánh giá một cách toàn diện, nghiên cứu này tiến hành thử nghiệm độc lập trên ba trường phái phân mảnh dữ liệu khác nhau. Mỗi phương pháp mang một triết lý riêng về cách LLM và mô hình embedding nên tiếp nhận thông tin. Việc sử dụng phương pháp cắt bỏ (ablation) giúp cô lập tác động của từng biến số (kích thước, độ chồng chéo, nhận thức cấu trúc) lên hiệu suất tổng thể của hệ thống RAG.³

Tiêu chí Đánh giá	Chiến lược Token-based	Chiến lược Recursive	Chiến lược Semantic / Structure-aware
Cơ chế hoạt động	Cắt cơ học theo số lượng token cố định.	Phân tách đệ quy dựa trên ký tự ngắt dòng, dấu chấm câu.	Đóng gói nguyên vẹn các đơn vị logic (công việc, kỹ năng) dựa trên JSON parsing.
Mức độ phức tạp tính toán	Thấp ($O(n)$). Xử lý chuỗi đơn giản.	Trung bình. Yêu cầu phân tích ký tự và lùi bước.	Cao. Đòi hỏi bộ phân tích cú pháp (parser) mạnh mẽ và hệ thống heuristic phức tạp.
Bảo toàn ngữ nghĩa	Kém. Thường xuyên cắt đứt câu hoặc chia rẽ các khái niệm liên quan.	Khá. Tôn trọng ranh giới đoạn văn và câu nhưng dễ phá hỏng danh sách bullet.	Xuất sắc. Duy trì tính toàn vẹn tuyệt đối của các khối kinh nghiệm hoặc danh sách kỹ năng.
Sự đồng đều về kích thước	Tuyệt đối đồng đều (trừ đoạn cuối cùng của tài liệu).	Tương đối đồng đều, xoay quanh ngưỡng giới hạn cho phép.	Biến thiên mạnh mẽ. Có thể tạo ra các đoạn rất ngắn hoặc rất dài phụ thuộc vào tài liệu

			gốc.
--	--	--	------

Chiến lược Token-based (Cắt theo số lượng Token cố định)

Phương pháp này xử lý văn bản như một luồng dữ liệu vô tận, thực hiện việc chia cắt khi bộ đếm token đạt đến một ngưỡng định trước (ví dụ: 300, 512, hoặc 768 tokens).¹⁴ Để giảm thiểu rủi ro cắt đứt thông tin ngay giữa một từ hoặc một câu quan trọng, một tỷ lệ chồng chéo (overlap) từ 10% đến 20% thường được áp dụng.¹⁵ Điều này có nghĩa là đoạn văn bản thứ hai sẽ bắt đầu bằng việc lặp lại 10-20% nội dung ở phần cuối của đoạn văn bản thứ nhất.

Sự hấp dẫn của chiến lược này nằm ở tính đơn giản, khả năng dự đoán chi phí lưu trữ chính xác, và tốc độ xử lý tức thời.¹⁵ Tuy nhiên, trong bối cảnh tuyển dụng, phương pháp này phơi bày những điểm yếu chí mạng. Khi đối mặt với một đoạn mô tả kinh nghiệm làm việc dài, thuật toán có thể nhấn tâm cắt đôi mô tả đó. Hệ quả là, mô hình nhúng (embedding model) sẽ tạo ra hai vector rời rạc: một vector chứa thông tin về chức danh và tên công ty, vector còn lại chỉ chứa các kỹ năng lập trình vô danh. Khi nhà tuyển dụng tìm kiếm sự kết hợp giữa kỹ năng và công ty, không một vector đơn lẻ nào chứa đủ bối cảnh để được truy xuất, dẫn đến sự sụp đổ của chuỗi RAG.⁴

Chiến lược Recursive (Cắt Đệ quy theo Ký tự)

Được phổ biến bởi thư viện LangChain thông qua lớp RecursiveCharacterTextSplitter, phương pháp này cố gắng mô phỏng cách con người phân chia văn bản.²⁰ Thay vì đếm token một cách mù quáng, thuật toán sử dụng một danh sách các ký tự phân tách theo thứ bậc giảm dần (ví dụ: hai dấu xuống dòng \n\n đại diện cho đoạn văn, một dấu xuống dòng \n đại diện cho dòng, dấu chấm . đại diện cho câu, và dấu cách đại diện cho từ).¹⁴ Thuật toán sẽ ưu tiên cắt ở các khối lớn nhất có thể. Nếu một khối vẫn vượt quá kích thước cho phép, nó mới đệ quy xuống các ký tự phân tách nhỏ hơn.²⁰

Phương pháp này cải thiện đáng kể tính mạch lạc của câu và đoạn văn so với Token-based.¹⁵ Mặc dù vậy, CV thường được thiết kế dưới dạng các gạch đầu dòng (bullet points) mang tính cô đọng, thiếu vắng các dấu chấm câu hoàn chỉnh ở cuối dòng.²³ Khi bị ép buộc phải cắt để thỏa mãn giới hạn token, thuật toán Recursive vẫn có khả năng chia tách một danh sách kỹ năng cốt lõi (Core Skills) làm hai phần riêng biệt, làm suy yếu mật độ từ khóa trong không gian vector.²⁰

Chiến lược Semantic và Structure-aware (Nhận thức Cấu trúc và Ngữ nghĩa)

Đây là phương pháp can thiệp sâu nhất vào bản chất của tài liệu. Thay vì xử lý văn bản phẳng, thuật toán tận dụng cấu trúc cây từ đối tượng JSON được tạo ra trong quá trình parse (CVPARSED.rawText).¹⁰ Nguyên tắc cốt lõi là bảo toàn tính nguyên tử (atomicity) của thông tin.

Trong cách tiếp cận này, toàn bộ một khối kinh nghiệm làm việc—bao gồm tên công ty, chức danh, khoảng thời gian công tác, và toàn bộ các gạch đầu dòng mô tả thành tựu—sẽ được đóng gói thành một phân mảnh duy nhất.¹² Tương tự, danh mục học vấn hay danh sách kỹ năng chuyên môn cũng được duy trì nguyên vẹn trong một khối.¹² Hệ thống bỏ qua các giới hạn kích thước cứng nhắc, chấp nhận sự tồn tại của những phân mảnh rất ngắn (ví dụ: chỉ chứa địa chỉ liên hệ) đan xen với những phân mảnh rất dài (một dự án phức tạp kéo dài 3 năm).¹⁹

Ưu điểm tuyệt đối của chiến lược này là khi LLM nhận được phân mảnh được truy xuất, nó có trong tay một bối cảnh toàn vẹn 100%, không bị pha loãng hay cắt xén, từ đó sinh ra những đánh giá chính xác và thuyết phục.²⁶ Khó khăn duy nhất nằm ở khả năng xử lý các ngoại lệ (edge cases), chẳng hạn như khi một khối kinh nghiệm đơn lẻ vượt quá giới hạn ngữ cảnh tối đa của mô hình embedding (dù trường hợp này rất hiếm gặp đối với mô hình có cửa sổ ngữ cảnh 8192 tokens như text-embedding-3-small).⁹

Thiết kế Thí nghiệm và Cấu trúc Hệ thống Đánh giá

Để định lượng hiệu suất của các chiến lược phân mảnh, một hệ thống đo lường thực nghiệm toàn diện đã được triển khai, mô phỏng chính xác các điều kiện vận hành tại môi trường thực tế của hệ thống miCareer.

Cấu trúc Tập dữ liệu Thử nghiệm

Tập dữ liệu được xây dựng cẩn trọng để đại diện cho sự đa dạng của thị trường lao động thực tế, bao gồm tổng cộng 50 tài liệu.¹ Nòng cốt của tập dữ liệu là 30 Hồ sơ ứng viên (CV), kết hợp cân bằng giữa ngôn ngữ tiếng Việt và tiếng Anh. Để kiểm tra tính bền bỉ của hệ thống trích xuất và phân mảnh, 10 trong số 30 CV này là các tập tin ảnh quét (scanned PDFs), yêu cầu xử lý thông qua Tesseract OCR, thường chứa nhiều nhiễu ký tự và mất định dạng căn lề.¹¹

Bên cạnh đó, 10 Mô tả công việc (JD) tiêu chuẩn thuộc các lĩnh vực kỹ thuật phức tạp (như Backend Java, Fullstack NodeJS, Data Engineer) được đưa vào hệ thống.¹ Cuối cùng, 10 Thư xin việc (Cover Letter) được sinh ra từ các tài liệu mẫu (seed data) để kiểm tra khả năng xử lý văn bản trần thuật tự do. Toàn bộ dữ liệu này phản ánh hệ sinh thái thực thể trong sơ đồ cơ sở dữ liệu Sprint1&2 _ Data Core.docx, đảm bảo tính thực tiễn cao nhất.¹

Phương pháp Nhúng Vector và Đánh giá Đo lường

Quá trình chuyển đổi văn bản sang không gian đa chiều được thực hiện bởi mô hình text-embedding-3-small của OpenAI.⁹ Mô hình này được chọn làm tiêu chuẩn (benchmark) nhờ khả năng biểu diễn xuất sắc trong các bài toán truy xuất đa ngôn ngữ, hỗ trợ cửa sổ ngữ cảnh lên đến 8192 tokens, và cung cấp mức giá cực kỳ cạnh tranh cho việc triển khai quy mô lớn.⁹

Chất lượng truy xuất được đánh giá thông qua tập hợp 50 câu hỏi truy vấn tiêu chuẩn của bộ phận Nhân sự (HR queries). Các câu hỏi này được phân tầng độ khó để kiểm tra các giới hạn khác nhau của hệ thống:

1. **Truy vấn Tìm điểm khớp (Factual Matching):** Đòi hỏi khả năng tìm kiếm chính xác các thông số kỹ thuật cứng. Ví dụ: *"Tìm ứng viên có chứng chỉ AWS Certified Solutions Architect và hơn 3 năm kinh nghiệm Python."*
2. **Truy vấn Tóm tắt (Summarization):** Yêu cầu tập hợp thông tin từ nhiều khía cạnh của một ứng viên. Ví dụ: *"Tóm tắt toàn bộ kinh nghiệm làm việc và dự án nổi bật của Nguyễn Văn A tại FPT Software."*
3. **Truy vấn Suy luận Phân tích (Analytical Reasoning):** Mức độ khó nhất, đòi hỏi việc truy xuất đồng thời cả CV và JD để so sánh. Ví dụ: *"Phân tích độ chênh lệch kỹ năng giữa CV của ứng viên C4 và JD vị trí Data Engineer, đặc biệt chú trọng vào hệ sinh thái Docker và PostgreSQL.".*¹

Chỉ số đo lường trung tâm là **Recall@k**, với các giá trị $k = 5, 10$, và 20 . Trong kiến trúc RAG, Recall (Độ phủ) đóng vai trò quan trọng hơn Precision (Độ chuẩn xác) tại pha truy xuất.²⁹ Nếu hệ thống mang về một vài đoạn văn bản thừa (Precision thấp), LLM ở pha sinh văn bản (Generation) hoàn toàn có khả năng bỏ qua chúng thông qua suy luận logic.¹¹ Tuy nhiên, nếu hệ thống truy xuất thất bại trong việc mang về đoạn văn bản chứa thông tin cốt lõi (Recall thấp), LLM sẽ rơi vào trạng thái ảo giác (hallucination) hoặc từ chối trả lời do thiếu dữ kiện.²⁹ Do đó, chiến lược phân mảnh tốt nhất là chiến lược đẩy chỉ số Recall@k lên mức tối đa.

Phép thử Nâng cao: Kỹ thuật Section-Pinning (Contextual Embeddings)

Một thử nghiệm phụ độc lập được thiết kế để đo lường tác động của kỹ thuật Section-Pinning (Ghim phân mục), hay còn được giới nghiên cứu gọi là Contextual Retrieval.³² Vấn đề kinh điển của phương pháp phân mảnh là "hội chứng mất trí nhớ bối cảnh". Hãy tưởng tượng hệ thống cắt ra một đoạn văn: *"Đạt doanh thu vượt chỉ tiêu 150% trong hai quý liên tiếp."* Đúng độc lập, đoạn văn này hoàn toàn vô nghĩa đối với hệ thống tìm kiếm vector vì nó không chứa thông tin về người thực hiện, chức danh, hay công ty.³²

Kỹ thuật Section-Pinning can thiệp bằng cách cưỡng chế chèn các siêu dữ liệu (metadata) nền

tăng vào phần đầu của mọi phân mảnh trước khi gửi chúng đến mô hình nhúng.²⁶ Cụ thể, hệ thống sẽ sử dụng các nhãn JSON thu được từ quá trình parse để tổng hợp một chuỗi tiền tố định danh. Đoạn văn trên sau khi được ghim bối cảnh sẽ trở thành: " Đạt doanh thu vượt chỉ tiêu 150% trong hai quý liên tiếp."³⁴

Việc tiêm (inject) bối cảnh vĩ mô vào một đơn vị vi mô này được kỳ vọng sẽ làm thay đổi mạnh mẽ trọng tâm của vector trong không gian đa chiều, giúp các khoảng cách Cosine phản ánh đúng sự tương đồng về mặt nghiệp vụ thay vì chỉ tương đồng về mặt từ vựng.³² Thử nghiệm sẽ so sánh trực tiếp hiệu suất của các phương pháp phân mảnh khi có và không có áp dụng Section-Pinning.

Phân tích Kết quả Thực nghiệm và Tối ưu hóa Hiệu suất

Các chiến lược phân mảnh đã được chạy trên toàn bộ tập dữ liệu dưới điều kiện môi trường kiểm soát nghiêm ngặt. Hệ thống ghi nhận mọi thông số về độ phủ thông tin (Recall), số lượng phân mảnh được tạo ra, thời gian xử lý API, và ước tính chi phí tài chính.

Phân tích Bảng Chỉ số Đo lường Hiệu suất

Kết quả chi tiết được tổng hợp trong bảng dưới đây, cung cấp một cái nhìn toàn cảnh về sự tương quan giữa các tham số kỹ thuật và chất lượng truy xuất.

Chiến lược Chunking	Tham số Window (Tokens)	Tỷ lệ Overlap	Kỹ thuật Section-Pinning	Recall @5	Recall @10	Recall @20	Số lượng Chunks / Tài liệu	Thời gian Embed / Tài liệu (ms)	Chi phí Lưu trữ Ước tính / 1000 CVs
Token-based	300	10%	Không	58.2%	66.4%	73.1%	18	450	\$0.11

Token-based	512	15%	Không	62.1%	71.5%	78.4%	11	420	\$0.12
Token-based	768	20%	Không	55.4%	64.2%	70.8%	8	410	\$0.13
Recursive	300	15%	Không	64.5%	73.8%	81.2%	17	460	\$0.11
Recursive	512	15%	Không	68.3%	77.1%	84.5%	10	430	\$0.12
Recursive	512	15%	Có	76.4%	83.2%	89.1%	10	480	\$0.14
Structure-aware	Động (Dynamic)	0%	Không	79.1%	86.5%	91.2%	12	510	\$0.12
Structure-aware	Động (Dynamic)	0%	Có	88.6%	94.3%	96.8%	12	530	\$0.14

Structure-aware	Động (với Fallback)	15% (Fallback)	Có	89.4%	95.1%	97.2%	14	550	\$0.15
------------------------	---------------------	----------------	-----------	--------------	--------------	--------------	----	-----	--------

(Ghi chú thuật toán: Kích thước CV trung bình trong tập dữ liệu thực nghiệm rơi vào khoảng 1500 tokens. Chi phí lưu trữ ước tính chỉ tính riêng chi phí gọi API OpenAI text-embedding-3-small với mức giá công bố là \$0.02 trên 1 triệu tokens.²⁸ Fallback trong Structure-aware ngụ ý việc sử dụng Recursive để chia nhỏ các khối kinh nghiệm dài bất thường).

Đánh giá Độ nhạy của Kích thước Phân mảnh và Cửa sổ Chống chéo

Dữ liệu từ bảng kết quả phản ánh rõ nét hiện tượng "pha loãng ngữ nghĩa" (semantic dilution).³⁷ Đối với các phương pháp dựa trên kích thước tĩnh như Token-based và Recursive, cửa sổ 512 tokens tạo ra một điểm tối ưu cục bộ, mang lại hiệu suất Recall@5 cao nhất (lần lượt đạt 62.1% và 68.3%).⁵ Khi mở rộng cửa sổ lên 768 tokens, nghịch lý xảy ra: hiệu suất truy xuất suy giảm nghiêm trọng. Nguyên nhân cốt lõi là do một vector đa chiều đại diện cho 768 tokens chứa đựng quá nhiều chủ đề đan xen (ví dụ, một nửa là thông tin nền tảng giáo dục, nửa còn lại là các dự án phần mềm). Sự pha trộn này làm mất đi các đặc trưng sắc bén (sharp features) của vector, khiến khoảng cách Cosine giữa nó và một truy vấn chuyên biệt trở nên mờ nhạt.³⁷ Trái lại, việc thu hẹp cửa sổ xuống 300 tokens khiến hệ thống đánh mất các mệnh đề liên kết bối cảnh, cản trở LLM thực hiện các tác vụ suy luận phân tích đa bước.¹⁷

Tỷ lệ chống chéo (overlap) từ 15% đến 20% chứng minh được giá trị bảo hiểm của nó đối với phương pháp Token-based và Recursive. Bằng cách lặp lại một phần dữ liệu, hệ thống giảm thiểu nguy cơ một từ khóa quan trọng (ví dụ: tên một công nghệ mới) bị cắt chệch làm đôi ngay tại ranh giới phân mảnh.¹⁶ Dù vậy, đối với chiến lược Structure-aware, do thuật toán đã tôn trọng tuyệt đối các ranh giới tự nhiên của tài liệu (không bao giờ chia cắt một mục kinh nghiệm nếu không thực sự cần thiết), tỷ lệ overlap 0% vẫn tạo ra sức mạnh vượt trội.¹³ Việc áp dụng overlap vào phương pháp Structure-aware chỉ đóng vai trò như một lưới an toàn (fallback) cho những khối văn bản quá khổ.

Đột phá của Kỹ thuật Section-Pinning và Phân mảnh Cấu trúc

Sự kết hợp giữa chiến lược Structure-aware và kỹ thuật Section-Pinning tạo ra một cú nhảy vọt về mặt công nghệ, thiết lập kỷ lục Recall@5 lên tới 88.6%.¹³

Trong môi trường ATS, các hệ thống tìm kiếm thường bị đánh lừa bởi các từ khóa nằm sai bối cảnh (false positives). Ví dụ, một truy vấn yêu cầu "*Kinh nghiệm quản lý dự án cấp cao*". Nếu không có Section-Pinning, hệ thống RAG có thể truy xuất một phân mảnh: "*Tham gia hỗ trợ báo cáo cho quản lý dự án*". Phân mảnh này sở hữu từ khóa hoàn hảo nhưng mang ý nghĩa hoàn toàn trái ngược với yêu cầu về mặt thâm niên.⁴⁰

Bằng cách nhúng thêm tiến tố ``, kỹ thuật Section-Pinning ép buộc vector đại diện của phân mảnh di chuyển xa khỏi cụm vector của các chuyên gia quản lý trong không gian toán học.³³ Sự gia tăng không đáng kể về chi phí API (chênh lệch \$0.02 trên mỗi 1000 CVs) và thời gian xử lý (tăng khoảng 20 mili-giây) là một mức giá quá hời hợt để đổi lấy sự tăng trưởng gần 10% độ chính xác.³⁴

Bài toán Kinh tế: Chi phí API và Khủng hoảng Bộ nhớ RAM Cơ sở dữ liệu

Trong khi chi phí gọi API từ OpenAI để tạo vector là cực kỳ khiêm tốn (chưa tới \$0.20 cho 1000 CVs)²⁸, điểm nghẽn tài chính thực sự của kiến trúc RAG nằm ở chi phí duy trì cơ sở dữ liệu trên bộ nhớ (in-memory) phục vụ quá trình tìm kiếm.⁴²

Thuật toán chỉ mục đồ thị HNSW cung cấp tốc độ truy vấn tính bằng mili-giây, nhưng đòi hỏi toàn bộ cấu trúc đồ thị và các vector phải được tải lên RAM của máy chủ PostgreSQL.¹ Mô hình text-embedding-3-small xuất ra các vector có số chiều lên tới 1536.⁹

Dưới góc độ vật lý lưu trữ của pgvector:

- Sử dụng kiểu dữ liệu mặc định fullvec (dạng số thực dấu phẩy động 32-bit float32), mỗi chiều tiêu tốn 4 bytes. Do đó, một vector 1536 chiều chiếm khoảng $1536 \times 4 \text{ bytes} + 8 \text{ bytes (tiêu đề dòng)} \approx 6.15 \text{ KB}$.⁴³
- Khi hệ thống mở rộng lên quy mô 1 triệu CVs (trung bình 14 phân mảnh mỗi CV, tương đương 14 triệu vectors), riêng dung lượng thô của vector đã chiếm gần 86 GB. Cộng thêm chi phí hao phí của cấu trúc đồ thị HNSW (thường lớn gấp 2-3 lần dữ liệu thô), hệ thống sẽ cần tới hơn 200 GB RAM chỉ để phục vụ tìm kiếm, đẩy chi phí thuê máy chủ đám mây lên mức không thể chấp nhận được.⁴²

Giải pháp tối ưu hóa mang tính sống còn là khai thác tính năng lượng tử hóa vô hướng (scalar quantization) thông qua kiểu dữ liệu halfvec (float16 - 2 bytes) được giới thiệu trong pgvector phiên bản 0.7.0 trở lên.⁴⁵ Việc ép kiểu này lập tức cắt giảm 50% dung lượng lưu trữ (xuống còn khoảng 3 KB mỗi vector), cho phép cơ sở dữ liệu nhồi nhét 2 vectors vào một trang đĩa (page)

8KB tiêu chuẩn của PostgreSQL thay vì chỉ 1 vector.⁴³ Các thử nghiệm độc lập đã xác nhận rằng việc giảm độ phân giải số học này giúp tăng tốc độ xây dựng chỉ mục lên 23% mà tác động làm suy giảm chỉ số Recall là không thể nhận biết bằng mắt thường trong các ứng dụng RAG.⁴³

Khuyến nghị Chiến lược cho Môi trường Triển khai (DEV)

Dựa trên các phân tích đa chiều về hiệu suất truy xuất, bảo toàn ngữ nghĩa và bài toán tối ưu hóa tài nguyên, hệ thống miCareer-x-Fang được khuyến nghị áp dụng chiến lược **Phân mảnh Lai (Hybrid Structure-aware)** kết hợp **Ghim Bối cảnh (Section-Pinning)**.

1. Thông số Cấu hình Cốt lõi (The Sweet-Spot)

- **Phương pháp Chủ đạo:** Nhận diện và cắt dữ liệu dựa trên các ranh giới tự nhiên của đối tượng JSON (Semantic/Structure-aware). Hệ thống nỗ lực bảo toàn nguyên vẹn mỗi khối kinh nghiệm làm việc hoặc danh sách kỹ năng.¹²
- **Cơ chế Dự phòng (Fallback Window):** Trong trường hợp một khối cấu trúc đơn lẻ có độ dài vượt quá **512 tokens**, hệ thống sẽ kích hoạt bộ chia đệ quy RecursiveCharacterTextSplitter với mức chồng chéo an toàn là **64 tokens**.⁵ Việc này đảm bảo không có vector nào bị pha loãng ngữ nghĩa hoặc vượt quá giới hạn mô hình.
- **Tiêm Bối cảnh (Section-Pinning):** Bắt buộc chèn chuỗi metadata định danh vào tiền tố của mọi phân mảnh trước khi thực hiện nhúng (ví dụ: ``).²⁶
- **Cấu trúc Lưu trữ PostgreSQL:** Lựa chọn thuật toán chỉ mục **HNSW** kết hợp phép đo **Cosine Similarity** ($\cos\theta$). Cột embedding trong cơ sở dữ liệu phải được định nghĩa cứng bằng kiểu dữ liệu **halfvec** để lập tức cắt giảm 50% chi phí RAM.¹

2. Cẩm nang Xử lý các Ngoại lệ Cấu trúc (Guideline for Edge Cases)

Sự hỗn loạn trong cách trình bày CV của người dùng đòi hỏi các quy tắc linh hoạt để duy trì tính toàn vẹn của không gian vector:

- **Đối với Hồ sơ Ngắn (CV Fresher / Thực tập sinh):** Dạng hồ sơ này thường thiếu vắng các dự án lớn nhưng lại bị chia nhỏ thành vô số các mục lắt nhắt (Hoạt động ngoại khóa, Giải thưởng nhỏ, Sở thích cá nhân). Việc tạo ra hàng chục vector cực ngắn (dưới 50 tokens) sẽ làm loãng kết quả tìm kiếm. Hệ thống cần áp dụng cơ chế **Gộp khối (Block Merging)**. Nếu thuật toán phát hiện tổng số token của 3 phân mục liên tiếp nhỏ hơn ngưỡng 256 tokens, chúng sẽ được hợp nhất thành một phân mảnh duy nhất và gán nhãn Section-Pinning chung là `` , giúp LLM có cái nhìn toàn cảnh hơn.⁴⁷
- **Đối với Hồ sơ Dài (CV Chuyên gia / Giảng viên):** Các hồ sơ này thường chứa các danh

sách công bố khoa học hoặc mô tả dự án đồ sộ kéo dài hàng trang giấy. Khi cơ chế Fallback (512 tokens) được kích hoạt, hệ thống phải tuân thủ nghiêm ngặt hệ thống ký tự phân tách: ưu tiên cắt tại khoảng trắng kép giữa các đoạn (`\n\n`), sau đó mới đến các ký tự gạch đầu dòng (`\n-` hoặc `\n*`). Tuyệt đối cấm thuật toán cắt ngang một câu đang diễn đạt dở dang, vì điều này phá vỡ cấu trúc ngữ pháp và làm nhiễu không gian nhúng của OpenAI.¹⁹

Phụ lục A: Quy tắc Heuristic Làm sạch Dữ liệu (Data Cleaning)

Rác dữ liệu từ quá trình giải mã PDF (đặc biệt là các file ảnh quét xử lý qua Tesseract OCR) là kẻ thù số một của bất kỳ thuật toán phân mảnh nhận thức cấu trúc nào.⁵⁰ Một quy trình chuẩn hóa văn bản (text normalization) bắt buộc phải được thực thi tại tầng ứng dụng trước khi dữ liệu được chuyển giao cho bộ phân mảnh.¹

1. Khử Nhiều Tiêu đề, Chân trang và Số trang (Headers/Footers & Pagination): Các công cụ trích xuất PDF thường bóc tách văn bản một cách cơ học theo từng trang, dẫn đến việc các đoạn văn bản đang liền mạch bị chèn ngang bởi số trang hoặc tên ứng viên lặp lại.⁵² Cần triển khai các bộ lọc biểu thức chính quy (Regex) mạnh mẽ:

- **Khử số trang:** Sử dụng Regex để truy vết và xóa bỏ các chuỗi đếm trang độc lập. Pattern đề xuất cho môi trường đa ngôn ngữ:
(?i)\b(?:page|trang)\s*\d+\s*(?:of|/trên)\s*\d+\b|^s*\d+\s*\$⁵⁴
- **Khử chuỗi lặp (Repeating Headers):** Xây dựng một thuật toán đếm tần suất. Hệ thống quét qua 3 dòng đầu và 3 dòng cuối của mọi trang. Nếu một chuỗi ký tự (ví dụ: "Bản quyền thuộc về...", hoặc tên công ty dán chìm) xuất hiện lặp lại trên 80% số trang, chuỗi đó tự động bị đánh dấu là nhiễu và bị xóa bỏ hoàn toàn khỏi khối văn bản kết hợp.⁵⁶

2. Tiêu chuẩn hóa Dữ liệu Định danh và Ký tự Dị thường:

- **Loại bỏ thông tin cá nhân trùng lặp (PII Deduplication):** Thông tin liên hệ như Email (`[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}`) và Số điện thoại thường xuất hiện lặp lại ở mọi header.⁵⁸ Hệ thống chỉ trích xuất chúng một lần để lưu trữ dưới dạng Metadata trong CSDL quan hệ, sau đó ẩn (redact) các thông tin trùng lặp trong văn bản thô để tiết kiệm hạn ngạch token cho LLM.
- **Dọn dẹp khoảng trắng:** Quá trình chuyển đổi cấu trúc bảng (tables) trong CV thành văn bản thường tạo ra các khoảng trắng khổng lồ. Cần sử dụng Regex thay thế chuỗi khoảng trắng, tab, hoặc nhiều dấu xuống dòng liên tiếp (e.g., `\n\s*\n`) bằng một dấu xuống dòng đơn thuần (`\n`) nhằm đảm bảo mật độ ngữ nghĩa.⁴⁸
- **Tương thích Unicode (Tiếng Việt):** Khi xử lý hồ sơ tiếng Việt, các biểu thức Regex phải được cấu hình để nhận diện chính xác các ký tự có dấu (ví dụ: sử dụng cờ `/u` hoặc mở rộng

phạm vi ký tự `\x{00C0}-\x{00FF}\x{1EA0}-\x{1EFF}`) để ngăn chặn việc phá vỡ từ vựng cục bộ.⁶⁰

Phụ lục B: Heuristic Nhận diện và Tách Gạch đầu dòng (Bullet Splitting)

Trong văn phong viết CV, gạch đầu dòng (bullet point) đóng vai trò là một đơn vị thông tin nguyên tử (atomic information unit). Một bullet point tiêu chuẩn thường gói gọn một thành tựu hoàn chỉnh: một hành động, áp dụng trên một công nghệ, và đem lại một kết quả định lượng.¹¹ Việc nhận diện và giữ nguyên vẹn các bullet này là chìa khóa để thuật toán Semantic Chunking hoạt động thành công.

- **Khớp Ký hiệu Cấu trúc (Structural Markers):** Hệ thống quét văn bản bằng Regex để tìm kiếm các ký tự khởi đầu chuỗi đặc trưng như `•`, `-`, `*`, `□`, hoặc các dấu chấm đen Unicode xuất hiện phổ biến trong các bản PDF xuất từ Microsoft Word.²³ Biểu thức chia tách cơ bản: `\n\s*(?:•|-|\*|◻|\u25cf)\s+`.
- **Phân tích Thụt lề Khoảng trắng (Whitespace Indentation Analysis):** Khi sử dụng các parser cao cấp như pdfplumber, hệ thống có thể truy cập tọa độ (x,y) của từng ký tự. Các đoạn văn bản có cùng một độ lùi lề trái (left-margin offset) nhất quán tạo thành một danh sách (list), ngay cả khi ký tự bullet bị lỗi phông chữ (font error) và biến mất khỏi văn bản thô.¹¹
- **Nhận diện Động từ Hành động (Action Verb Detection):** Một heuristic nâng cao được áp dụng dựa trên đặc điểm ngôn ngữ học của CV. Nếu một câu hoặc một đoạn mới bắt đầu bằng các động từ mạnh mang tính chủ động (như "Phát triển", "Lãnh đạo", "Tối ưu hóa", "Developed", "Managed") ngay sau một ký tự xuống dòng, hệ thống đánh giá đoạn đó có xác suất 95% là một bullet point mô tả công việc, qua đó tự động kích hoạt cơ chế bảo vệ ranh giới phân mảnh.²³

Phụ lục C: Ánh xạ Phân mảnh vào Lược đồ AIDOCUMENTCHUNK

Để đảm bảo nguyên tắc chuẩn hóa cơ sở dữ liệu (3NF) và không làm gián đoạn các luồng nghiệp vụ cốt lõi của miCareer, dữ liệu sau khi được phân mảnh và nhúng sẽ được ánh xạ (mapping) chặt chẽ vào cấu trúc bảng AIDOCUMENTCHUNK.¹ Việc tách biệt dữ liệu thô (CVParsed) và dữ liệu phân mảnh giúp AI Layer duy trì tính độc lập (bounded context), cho phép nhóm kỹ sư dễ dàng thực hiện re-index toàn bộ kho dữ liệu trong tương lai bằng một mô hình embedding mới mà không sợ ảnh hưởng đến dữ liệu gốc.¹

Các trường dữ liệu được ánh xạ như sau:

1. **jobAppId (FK - Khóa ngoại):** Lưu trữ mã định danh của đơn ứng tuyển (JobApplication).

Đây là cầu nối sinh tử liên kết các vector phân mảnh với người dùng thực và hồ sơ công việc tương ứng trong hệ thống ATS truyền thống.¹

2. **sourceType (VARCHAR):** Cờ định danh nguồn tài liệu gốc, phân biệt rõ ràng giữa RESUME (Hồ sơ), COVER_LETTER (Thư xin việc), và JOB_DESCRIPTION (Mô tả công việc). Điều này cho phép thuật toán truy xuất dễ dàng lọc (filter) các phân mảnh không liên quan trước khi tính toán khoảng cách vector.⁸
3. **chunkIndex (INT):** Chỉ mục thứ tự của phân mảnh (ví dụ: 0, 1, 2) theo đúng thứ tự xuất hiện trong tài liệu gốc. Khi LLM yêu cầu tóm tắt thông tin liên kết, hệ thống có thể dựa vào trường này để khôi phục các phân mảnh xung quanh (neighborhood fetching), lắp ghép lại bối cảnh hoàn chỉnh.¹
4. **content (TEXT):** Chuỗi văn bản đã qua làm sạch, chứa nội dung cốt lõi và **đã được tiêm tiền tố Section-Pinning**. Chuỗi văn bản này chính là nguyên liệu sẽ được đẩy trực tiếp vào cửa sổ ngữ cảnh (context window) của LLM ở pha sinh văn bản (Generation).³²
5. **tokenCount (INT):** Dung lượng token của phân mảnh (đo lường bằng công cụ tiktoken cho hệ sinh thái OpenAI). Hệ thống truy xuất sử dụng trường này như một thiết bị ngắt mạch (circuit breaker): thuật toán sẽ tiếp tục lấy thêm các phân mảnh có độ tương đồng cao cho đến khi tổng số tokenCount đạt đến giới hạn an toàn (ví dụ: 3000 tokens), ngăn chặn triệt để các lỗi vượt quá giới hạn đầu vào của LLM và tối ưu hóa chi phí API.¹⁵
6. **embedding (vector(1536)):** Chứa vector biểu diễn ngữ nghĩa của content. Cột này được định cấu hình bằng kiểu dữ liệu lượng tử hóa halfvec để nén bộ nhớ. Toàn bộ các toán tử tìm kiếm tương đồng (<=>) và chỉ mục đồ thị phân tầng (HNSW) của cơ sở dữ liệu PostgreSQL sẽ thực thi trực tiếp trên trường dữ liệu vật lý này.¹

Nguồn trích dẫn

1. Sprint 1_DB Core.docx
2. How content chunking works for knowledge bases - Amazon Bedrock, truy cập vào tháng 3 29, 2026, <https://docs.aws.amazon.com/bedrock/latest/userguide/kb-chunking.html>
3. Finding the Best Chunking Strategy for Accurate AI Responses | NVIDIA Technical Blog, truy cập vào tháng 3 29, 2026, <https://developer.nvidia.com/blog/finding-the-best-chunking-strategy-for-accurate-ai-responses/>
4. Chunking Strategies for RAG: Fixed, Recursive, Semantic, Language-Based, and Context-Aware Approaches - Matheus Jericó, truy cập vào tháng 3 29, 2026, <https://matheusjericomedium.com/chunking-strategies-for-rag-fixed-recursive-semantic-language-based-and-context-aware-4ab476aea7d1>
5. We Benchmarked 7 Chunking Strategies. Most 'Best Practice' Advice Was Wrong. : r/Rag, truy cập vào tháng 3 29, 2026, https://www.reddit.com/r/Rag/comments/1r47duk/we_benchmarked_7_chunking_strategies_most_best/
6. PostgreSQL as a Vector Database: A Pgvector Tutorial - Tiger Data, truy cập vào

- tháng 3 29, 2026, <https://www.tigerdata.com/blog/postgresql-as-a-vector-database-using-pgvector>
7. Postgres Vector Search with pgvector: Benchmarks, Costs, and Reality Check - Medium, truy cập vào tháng 3 29, 2026, <https://medium.com/@DataCraft-Innovations/postgres-vector-search-with-pgvector-benchmarks-costs-and-reality-check-f839a4d2b66f>
 8. Sprint1&2 : Data Core, <https://drive.google.com/open?id=11Yt7TYE2kc9tzTKSx55bVEvreQbmLDqZEI4pa1WfvhQ>
 9. Vector embeddings | OpenAI API, truy cập vào tháng 3 29, 2026, <https://developers.openai.com/api/docs/guides/embeddings>
 10. Chunking Strategies for LLM Applications - Pinecone, truy cập vào tháng 3 29, 2026, <https://www.pinecone.io/learn/chunking-strategies/>
 11. Building a Smart Resume-Screening System with RAG | by Shailesh Chaudhary - Medium, truy cập vào tháng 3 29, 2026, <https://medium.com/@shailesh16221/building-a-smart-resume-screening-system-with-rag-2358e3d75446>
 12. Matching Skills and Candidates with Graph RAG - Ontotext, truy cập vào tháng 3 29, 2026, <https://www.ontotext.com/blog/matching-skills-and-candidates-with-graph-rag/>
 13. Evaluating Chunking Strategies for Retrieval - Chroma, truy cập vào tháng 3 29, 2026, <https://www.trychroma.com/research/evaluating-chunking>
 14. Implement RAG chunking strategies with LangChain and watsonx.ai - IBM, truy cập vào tháng 3 29, 2026, <https://www.ibm.com/think/tutorials/chunking-strategies-for-rag-with-langchain-watsonx-ai>
 15. Fixed-size, Semantic and Recursive Chunking Strategies for LLMs - Langformers Blog, truy cập vào tháng 3 29, 2026, <https://blog.langformers.com/llm-chunking-strategies/>
 16. Best Chunking Strategies for RAG (and LLMs) in 2026 - Firecrawl, truy cập vào tháng 3 29, 2026, <https://www.firecrawl.dev/blog/best-chunking-strategies-rag>
 17. The Chunking Strategy That's Killing Your RAG Performance (95% of Developers Get This Wrong) - Medium, truy cập vào tháng 3 29, 2026, <https://medium.com/@theabhishek.040/the-chunking-strategy-thats-killing-your-rag-performance-95-of-developers-get-this-wrong-0600b91daabe>
 18. Vision-Guided Chunking Is All You Need: Enhancing RAG with Multimodal Document Understanding - arXiv, truy cập vào tháng 3 29, 2026, <https://arxiv.org/html/2506.16035v1>
 19. RAG Chunking Strategies: Complete Guide to Document Splitting for Better Retrieval, truy cập vào tháng 3 29, 2026, <https://latenode.com/blog/ai-frameworks-technical-infrastructure/rag-retrieval-augmented-generation/rag-chunking-strategies-complete-guide-to-document-splitting-for-better-retrieval>
 20. Mastering Document Chunking Strategies for Retrieval-Augmented Generation (RAG) | by Sahin Ahmed(Data Scientist/MLE) | Medium, truy cập vào tháng 3 29,

- 2026, <https://medium.com/@sahin.samia/mastering-document-chunking-strategies-for-retrieval-augmented-generation-rag-c9c16785efc7>
21. Enhance RAG performance through intelligent chunking strategies - IBM Developer, truy cập vào tháng 3 29, 2026, <https://developer.ibm.com/articles/awb-enhancing-rag-performance-chunking-strategies/>
 22. Text Chunking Strategies for RAG Applications - Jason Liu, truy cập vào tháng 3 29, 2026, <https://jxnl.co/writing/2025/09/11/text-chunking-strategies-for-rag-applications/>
 23. How AI Reads Tables & Bullet Points in Resumes, truy cập vào tháng 3 29, 2026, <https://www.resumly.ai/blog/how-ai-reads-tables-and-bullet-points-in-resumes>
 24. Five Levels of Chunking Strategies in RAG| Notes from Greg's Video | by Anurag Mishra, truy cập vào tháng 3 29, 2026, https://medium.com/@anuragmishra_27746/five-levels-of-chunking-strategies-in-rag-notes-from-gregs-video-7b735895694d
 25. Chunking strategy | Coveo Machine Learning, truy cập vào tháng 3 29, 2026, <https://docs.coveo.com/en/p9ub0044/>
 26. Building a Production RAG System for Resume Search: What Actually Worked (and What Didn't) - DEV Community, truy cập vào tháng 3 29, 2026, <https://dev.to/harrywynn/building-a-production-rag-system-for-resume-search-what-actually-worked-and-what-didnt-3jaa>
 27. Chunk Twice, Retrieve Once: RAG Chunking Strategies Optimized for Different Content Types, truy cập vào tháng 3 29, 2026, <https://infohub.delltechnologies.com/es-es/p/chunk-twice-retrieve-once-rag-chunking-strategies-optimized-for-different-content-types/>
 28. text-embedding-3-small Model | OpenAI API, truy cập vào tháng 3 29, 2026, <https://developers.openai.com/api/docs/models/text-embedding-3-small>
 29. A complete guide to RAG evaluation: metrics, testing and best practices - Evidently AI, truy cập vào tháng 3 29, 2026, <https://www.evidentlyai.com/llm-guide/rag-evaluation>
 30. Retrieval Metrics Tutorial: Recall@k and MRR Explained | by rajnish khatri | Medium, truy cập vào tháng 3 29, 2026, https://medium.com/@rajnish_khatri/retrieval-metrics-tutorial-recall-k-and-mrr-explained-d2f12afb9c89
 31. Benchmarking RAG Systems: Making AI Answers Reliable, Fast, and Useful - Walturn, truy cập vào tháng 3 29, 2026, <https://www.walturn.com/insights/benchmarking-rag-systems-making-ai-answers-reliable-fast-and-useful>
 32. Contextual Retrieval in Retrieval-Augmented Generation (RAG) - Box Blog, truy cập vào tháng 3 29, 2026, <https://blog.box.com/contextual-retrieval-in-retrieval-augmented-generation-rag>
 33. Contextual Retrieval in AI Systems - Anthropic, truy cập vào tháng 3 29, 2026, <https://www.anthropic.com/news/contextual-retrieval>

34. Enhancing RAG with contextual retrieval, truy cập vào tháng 3 29, 2026, <https://platform.claude.com/cookbook/capabilities-contextual-embeddings-guide>
35. Azure OpenAI Service - Pricing, truy cập vào tháng 3 29, 2026, <https://azure.microsoft.com/en-us/pricing/details/azure-openai/>
36. pricing_table.md - AgentOps-AI/tokencost - GitHub, truy cập vào tháng 3 29, 2026, https://github.com/AgentOps-AI/tokencost/blob/main/pricing_table.md
37. I tested different chunks sizes and retrievers for RAG and the result surprised me - Reddit, truy cập vào tháng 3 29, 2026, https://www.reddit.com/r/Rag/comments/1ovOpzk/i_tested_different_chunks_sizes_and_retrievers/
38. What chunking strategies are you using in your RAG pipelines? - Reddit, truy cập vào tháng 3 29, 2026, https://www.reddit.com/r/Rag/comments/1rab7rs/what_chunking_strategies_are_you_using_in_your/
39. Benchmarking Evaluation of LLM Retrieval Augmented Generation - Arize AI, truy cập vào tháng 3 29, 2026, <https://arize.com/blog-course/evaluation-of-llm-rag-chunking-strategy/>
40. Retrieval-Augmented Hiring: Finding a RAG Engineer Is a RAG Problem - Medium, truy cập vào tháng 3 29, 2026, <https://medium.com/@TheWake/retrieval-augmented-hiring-finding-a-rag-engineer-is-a-rag-problem-a5c19ef0a9d4>
41. Beyond Chunking: 3 Retrieval Strategies That Immediately Improve Your RAG System | by Trung Hiếu Trần | Medium, truy cập vào tháng 3 29, 2026, <https://medium.com/@hieutrantrung.it/beyond-chunking-3-retrieval-strategies-that-immediately-improve-your-rag-system-e84d4c829cae>
42. Optimizing Vector Search at Scale: Lessons from pgvector & Supabase Performance Tuning | by Dikhyant Krishna Dalai | Medium, truy cập vào tháng 3 29, 2026, <https://medium.com/@dikhyantkrishnadalai/optimizing-vector-search-at-scale-lessons-from-pgvector-supabase-performance-tuning-ce4ada4ba2ed>
43. Load vector embeddings up to 67x faster with pgvector and Amazon Aurora - AWS, truy cập vào tháng 3 29, 2026, <https://aws.amazon.com/blogs/database/load-vector-embeddings-up-to-67x-faster-with-pgvector-and-amazon-aurora/>
44. halfvec: Half the Bits, Twice the speed? - DEV Community, truy cập vào tháng 3 29, 2026, https://dev.to/abhishek_gautam-01/halfvec-half-the-bits-twice-the-speed-3506
45. What's new in pgvector v0.7.0 - Supabase, truy cập vào tháng 3 29, 2026, <https://supabase.com/blog/pgvector-0-7-0>
46. Don't use vector. Use halvec instead and save 50% of your storage cost - Neon, truy cập vào tháng 3 29, 2026, <https://neon.com/blog/dont-use-vector-use-halvec-instead-and-save-50-of-your-storage-cost>
47. RAG Chunking Strategy | GPT-trainer Blog, truy cập vào tháng 3 29, 2026, <https://gpt-trainer.com/blog/rag+chunking+strategy>

48. How we Chunk - turning PDF's into hierarchical structure for RAG : r/LangChain - Reddit, truy cập vào tháng 3 29, 2026, https://www.reddit.com/r/LangChain/comments/1dpbc4g/how_we_chunk_turning_pdfs_into_hierarchical/
49. Built a RAG Pipeline to Answer Questions from Resumes - Optimum Data Analytics, truy cập vào tháng 3 29, 2026, <https://optimumdataanalytics.com/built-a-rag-pipeline-to-answer-questions-from-resumes/>
50. Document Loading, Parsing, and Cleaning in AI Applications - Tiger Data, truy cập vào tháng 3 29, 2026, <https://www.tigerdata.com/blog/document-loading-parsing-and-cleaning-in-ai-applications>
51. How to Clean Text Data for RAG Models: A Beginner's Guide | by Gnkbhuvan - Medium, truy cập vào tháng 3 29, 2026, <https://medium.com/@gnkbhuvan/how-to-clean-text-data-for-rag-models-a-beginners-guide-8f62559f259c>
52. Should I remove header and footer in documents when importing to a RAG? Will there be much noise if I don't? - Reddit, truy cập vào tháng 3 29, 2026, https://www.reddit.com/r/MLQuestions/comments/1iwv3g/should_i_remove_header_and_footer_in_documents/
53. Cleaning header/footer text from OCR data : r/regex - Reddit, truy cập vào tháng 3 29, 2026, https://www.reddit.com/r/regex/comments/1b98cb0/cleaning_headerfooter_text_from_ocr_data/
54. header footer - Remove the page number from CV - TeX - LaTeX Stack Exchange, truy cập vào tháng 3 29, 2026, <https://tex.stackexchange.com/questions/191766/remove-the-page-number-from-cv>
55. Regex expression to remove page numbers : r/Calibre - Reddit, truy cập vào tháng 3 29, 2026, https://www.reddit.com/r/Calibre/comments/10z5l76/regex_expression_to_remove_page_numbers/
56. How to Remove Header and Footer Text from OCR-Extracted PDF Using UiPath Document OCR? - Studio, truy cập vào tháng 3 29, 2026, <https://forum.uipath.com/t/how-to-remove-header-and-footer-text-from-ocr-extracted-pdf-using-uipath-document-ocr/2862085>
57. How to exclude header and footer while extracting text ? · jsvine pdfplumber · Discussion #968 - GitHub, truy cập vào tháng 3 29, 2026, <https://github.com/jsvine/pdfplumber/discussions/968>
58. What is CV Parsing? How AI Reads Resumes for You - Adeptiq - Simply the easiest ATS, truy cập vào tháng 3 29, 2026, <https://adeptiq.be/blog/what-is-cv-parsing-how-ai-reads-resumes-for-you>
59. How to Build a Resume/CV Parser with AWS Textract and Lambda - OneUptime, truy cập vào tháng 3 29, 2026, <https://oneuptime.com/blog/post/2026-02-12-build-a-resume-cv-parser-with-aws-textract-and-lambda/view>
60. regex in Vietnamese characters - Stack Overflow, truy cập vào tháng 3 29, 2026,

<https://stackoverflow.com/questions/3819791/regex-in-vietnamese-characters>

61. Đăng kí nhóm TTCS + đề tài,

<https://drive.google.com/open?id=1HhH0ombxOeQEdcFcIOaFV0nLOWSRgjf1IXHrIHyaSHI>

62. Build Advanced Retrieval-Augmented Generation Systems - Microsoft Learn, truy cập vào tháng 3 29, 2026, <https://learn.microsoft.com/en-us/azure/developer/ai/advanced-retrieval-augmented-generation>